PROJETO DE DISPOSITIVOS MÓVEIS

Menos perdas. Mais futuro.

Uma aplicação mobile para dar visibilidade aos produtos próximos do vencimento em pequenos mercados.

pv

Validade vira prioridade.

React Native + Expo | BFF Node.js | SQLite

INTEGRANTES

Lucas Cabrio - 04723-133

Paulo Esteves - 04723-090

Projeto acadêmico • 27 de outubro de 2026
Empresa de referência fictícia: Mercado do Bairro.
Código modularizado, persistência real e integração externa.

01 / PROPÓSITO

O problema de negócio

Pequenos mercados precisam conferir a validade de vários produtos. Anotações dispersas dificultam identificar quais lotes exigem ação imediata. Isso pode gerar descarte e perda de oportunidades de venda.

Público e proposta

Dono ou funcionário responsável pela prateleira. O app reúne os lotes em uma lista por validade, exibe indicadores e recomenda descontos para os produtos ainda válidos.

O diferencial, sem complicar

Uma fila de prioridade com três decisões claras: acompanhar, considerar desconto ou retirar da venda. A consulta por código de barras evita redigitar o nome quando o produto existe no catálogo externo.

Regras implementadas no BFF

• Vencido: sinalizar retirada; nenhum desconto sugerido.

• De hoje até 2 dias: sugestão de 30% de desconto.

• De 3 a 7 dias: sugestão de 15% de desconto.

• Acima de 7 dias: acompanhar, sem desconto.

Benefícios esperados

Menos esquecimentos, cadastro mais rápido e decisões mais consistentes. Não houve piloto em empresa real; economia ou redução de perdas ainda não foram mensuradas.

Valor em risco: quantidade × preço dos lotes com validade entre hoje e 7 dias. É valor potencial de venda, não prejuízo realizado. Produtos vencidos são contabilizados separadamente.

02 / ARQUITETURA

Três camadas, um fluxo

APP • React Native + Expo

Telas, componentes, rotas e estado de sessão

HTTP JSON + token de sessão

↓

BFF • Node.js

Autenticação, validação, indicadores e descontos

SQL parametrizado

HTTPS + timeout

SQLite

Usuários, sessões e lotes

Open Food Facts

Consulta de nome por código

O aplicativo não abre o banco. Toda leitura e gravação passa pelo BFF. O backend também consulta a API externa e devolve somente os campos necessários à interface.

Por que BFF? Ele não apenas repassa registros: calcula validade, status, desconto e indicadores para este app. SQLite mantém o projeto simples e persiste em server/data.db, mesmo após reiniciar o servidor.

03 / ORGANIZAÇÃO E DADOS

Código fácil de explicar

Rotas e telas

mobile/src/app contém as rotas Expo Router e a proteção de acesso. screens contém LoginScreen, DashboardScreen, InventoryScreen e BatchFormScreen.

Componentes reutilizáveis

components reúne Button, Field, Message, Page e BatchCard. theme.ts centraliza cores e formatação. Cada componente tem uma responsabilidade visível.

Estado e integração

AuthContext gerencia sessão. InventoryContext guarda lotes e indicadores. services/api.ts centraliza fetch, token, timeout e tratamento de erros.

Backend separado por responsabilidade

app.js define rotas HTTP; auth.js cuida de senhas e sessões; rules.js valida e calcula; database.js cria o schema; seed.js prepara dados fictícios.

Modelo relacional

users: id, name, email único, password (hash).
sessions: token (hash), user_id, expires.
batches: id, user_id, name, barcode, quantity, price, expires, category, created_at.

Relacionamentos: um usuário possui várias sessões e vários lotes. Cada lote pertence a um usuário. O mesmo produto pode aparecer em lotes distintos, com validades diferentes.

04 / AUTENTICAÇÃO E DASHBOARD

Da entrada à decisão



1. Acesso: cadastro de conta, validação de campos e login. Senhas usam scrypt com salt; sessão opaca expira em 24 horas.



2. Visão geral: valor em risco, unidades, lotes e vencidos. Dados demonstrativos são fictícios e calculados pelo BFF.

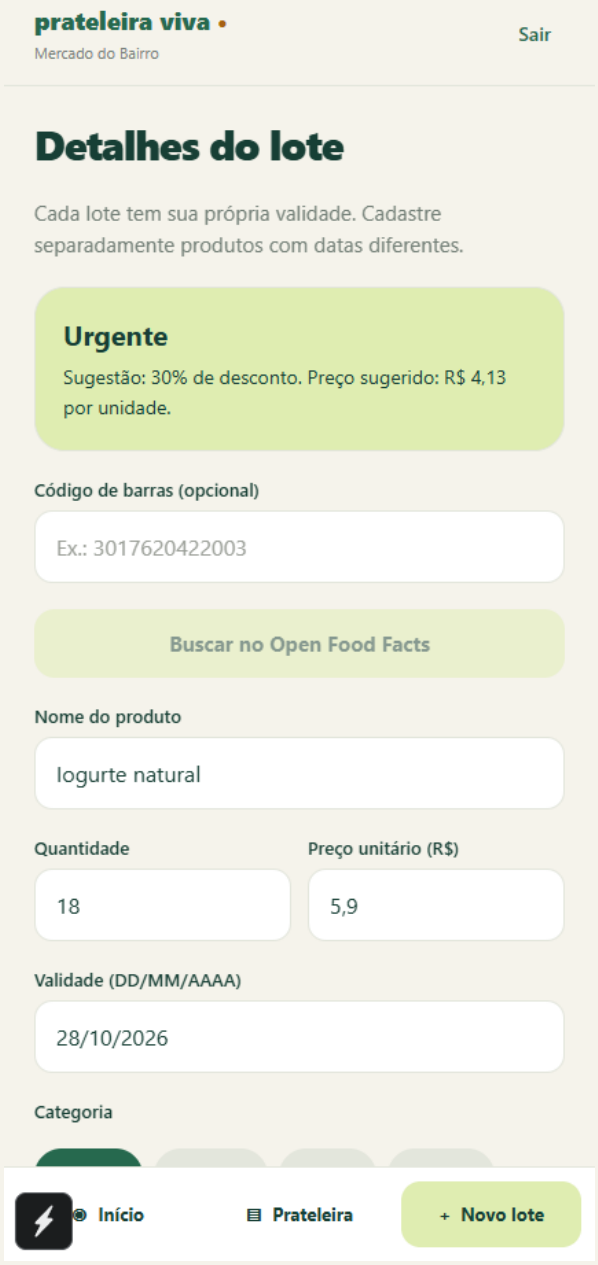
Capturas reais: Expo web, viewport 430 × 932. Não são prints de aparelho nativo.

05 / CONSULTA E EDIÇÃO

A prateleira em ordem



3. Consulta: busca por nome, filtro por prazo e ordenação por validade. Cada cartão abre os detalhes do lote.

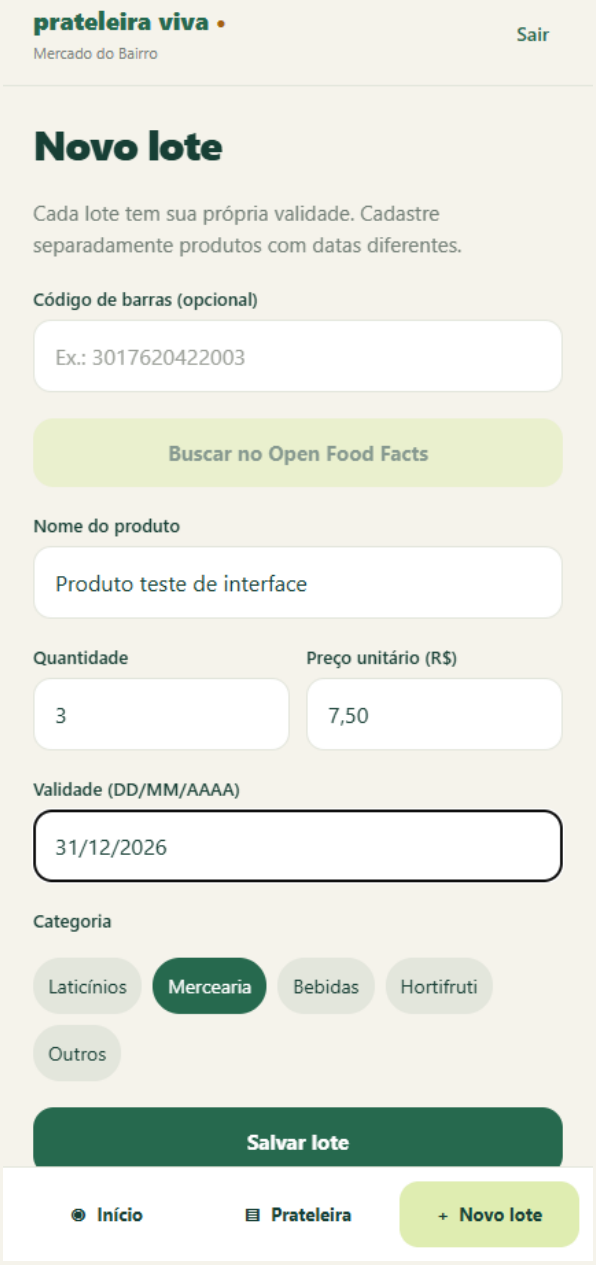


4. Edição: o formulário reaproveitado mostra o status e a sugestão de preço. Mudanças são validadas e persistidas.

Capturas reais da mesma aplicação React Native executada no navegador via Expo.

06 / CADASTRO E EXCLUSÃO

Fluxo completo de dados



5. Cadastro: nome, quantidade, preço, validade e categoria. Código de barras é opcional; consulta via BFF ao Open Food Facts.



6. Exclusão: confirmação explícita antes de apagar. A interface informa o resultado e atualiza a consulta.

O registro temporário usado no teste foi excluído; a conta demo foi preservada.

07 / API E SEGURANÇA

Integrações com propósito

Rotas principais

POST /auth/register e /auth/login: criar conta e autenticar.
GET /auth/me; POST /auth/logout: restaurar e revogar sessão.
GET /dashboard: indicadores calculados.
GET e POST /batches: consulta e cadastro.
PUT e DELETE /batches/:id: edição e exclusão.
GET /products/lookup?barcode=...: busca externa autenticada.

Open Food Facts

O BFF faz uma requisição HTTPS para /api/v2/product/{codigo}.json e solicita product_name e brands. O usuário revisa o nome recebido e informa os dados do lote. Teste real: código 3017620422003 retornou Nutella. Não é usado scanner de câmera.

Se não houver produto, retorna 404; em falha ou timeout, 502. A mensagem orienta o preenchimento manual. Os dados são colaborativos e a origem é identificada na tela. Documentação: openfoodfacts.github.io/openfoodfacts-server/api/

Controle de acesso e validação

Todas as rotas de negócio exigem Bearer token. O banco guarda o hash do token; o cliente nativo usa SecureStore e a versão web usa sessionStorage. Consultas verificam user_id para impedir acesso aos lotes de outra conta. Logout revoga a sessão.

Validação no cliente melhora a experiência; validação no servidor protege os dados. O backend rejeita quantidade inválida, preço não positivo, data inexistente e categoria desconhecida. SQL parametrizado evita construir consultas com valores digitados.

08 / EXECUÇÃO E VALIDAÇÃO

Preparado para demonstrar

Como executar

Requisito: Node.js 24 LTS.
1. Na raiz: **npm --prefix mobile ci**
2. Preparar demonstração: **npm run seed**
3. Primeiro terminal: **npm run backend**
4. Segundo terminal: **npm run web**
Conta: demo@prateleiraviva.com / Demo@12345

Para celular, configure EXPO_PUBLIC_API_URL com o IP do computador e mantenha ambos na mesma rede. Use Expo Go compatível com SDK 58 ou development build. Instruções completas e perfil EAS estão no README.

O que foi verificado

Testes HTTP: login, validação, CRUD, isolamento entre contas, expiração, logout, API externa simulada em sucesso/falha e persistência após reabrir o banco. Teste de interface: login até exclusão e bloqueio de rota após logout. Consulta externa real também realizada.

TypeScript e lint direto aprovados. Expo Doctor: 20/20 verificações. Bundles Android, iOS e web gerados. As telas foram testadas no navegador em dimensões de celular; não houve execução em dispositivo nativo nem geração de APK/IPA.

Limites e próximos passos

Escopo acadêmico local: sem vendas, notificações, modo offline ou equipe compartilhada. npm audit aponta 3 alertas high na cadeia Expo CLI/Zod; a correção automática sugerida reduz a versão do Expo e não foi aplicada. Antes de produção: revisar dependências, adotar HTTPS, restringir CORS, configurar backup e testar aparelhos.

09 / APRESENTAÇÃO

O que cada integrante explica

Lucas Cabrio - 04723-133

Apresentar o problema do mercado, o público e os benefícios esperados. Demonstrar login, indicadores, consulta e cadastro com busca por código de barras. Explicar os componentes reutilizados no frontend.

Paulo Esteves - 04723-090

Demonstrar edição, persistência ao reiniciar o backend e exclusão. Explicar BFF, banco, autenticação e regras de validade. Mostrar rules.js e database.js como exemplos centrais.

Perguntas para ensaiar

Por que SQLite? Persistência relacional simples para um protótipo local.
Por que BFF? Prepara dados, aplica regras e protege o acesso ao banco.
Se a API cair? Mensagem clara e cadastro manual.
O desconto é automático? Não, é uma sugestão para decisão do responsável.
Como comprovar persistência? Reiniciar o BFF e consultar o mesmo lote.

Referências e entregáveis

Código-fonte: github.com/LucasCabrio/prateleira-viva (privado).
Guia detalhado: README.md.
Roteiro: docs/roteiro-apresentacao.md.
Prints: docs/screenshots/.

Expo SDK 58: docs.expo.dev/versions/v58.0.0/
Expo Router: docs.expo.dev/router/installation/
SecureStore: docs.expo.dev/versions/v58.0.0/sdk/securestore/
Open Food Facts: openfoodfacts.github.io/openfoodfacts-server/api/
Node.js SQLite: nodejs.org/api/sqlite.html